

How CSS Works

Course Overview (5m)

Course Content

- Welcome: who am I?
- Overview: what can you expect to learn?
- Prerequisites: what should you already know?

Notes

Why Do So Many Developers Struggle With CSS?

We're going to kick off with a question: why do so many developers struggle with CSS? I've worked with a lot of really talented, capable developers in my career, proper full stack developers who were as happy working with SQL databases as they were writing HTML, and over the years it's noticeable how many of them really didn't enjoy working with stylesheets. We'd be working through a new feature - database schema migrations, new classes, backend, HTML, JavaScript, all absolutely fine, and then when it comes to styling the frontend... that was always the bit that nobody wanted to pick up.

There are folks out there who will tell you CSS isn't even a programming language. The best way to deal with those people is ask them to draw a heading in 72 point dark blue Times New Roman using C++ and come back when they've done it. CSS absolutely is a programming language - it's a structured language, with a formal grammar, that we can use to control part of a computer system. But CSS, on it's own, doesn't do anything. You can't write an app using 100% CSS - CSS only works when you combine it with HTML, which means it's often treated as an "afterthought" - first you make it work, THEN make it look nice. But, as you'll see over the next few hours, CSS should never be an afterthought, and there is way more to it than just making websites look nice.

I think there's a few more reasons why so many developers don't enjoy working with CSS.

First: CSS isn't procedural, it's declarative. With languages like JavaScript or

C#, you can read the code by starting at the beginning and following a series of instructions - do this, then do that, then do that. Of course, it's not always quite that easy, but that procedural style means you can pause the program halfway through, attach a debugger, figure out exactly what's going on right now - it's step-by-step, like following a recipe. CSS is more like ordering takeaway food: you just specify what you want, and when it works? Awesome. But when it doesn't work, you've got to work out why - and that could mean unpicking dozens of different CSS rules defined across different files and modules, figuring out which ones are applicable, which rules override other rules, how those rules interact with whatever else is on the page - and that's hard. In this course, I'm going to show you a bunch of techniques I've learned over the course of my own career for debugging and troubleshooting CSS, and how you can use those to make sense of what's going on.

Second: I think our industry has, historically, not done a great job teaching CSS as an engineering discipline. There's a lot of great books and courses out there about CSS, but many of them are aimed at designers - folks who work with Photoshop or Figma, who understand the principles of graphic design, layout, colours and typography. You don't need to be a great designer to be a great developer - some of the best developers I've worked with were absolutely terrible when it came to visual design - but you do need to understand the capabilities of the tech you're working with.

Third: CSS is hard to test. It doesn't crash. It usually doesn't produce error messages. If you accidentally change a heading from being 100 pixels high to being 10 pixels high, it's not going to break the build; your code still works, it just looks wrong - and unless you pick it up during manual testing, you're not going to know about that until somebody files a bug saying "the website looks weird".

Fourth: many parts of CSS are inconsistent, and some bits of it are just plain wrong. There's a wonderful page up on the CSS working group's wiki called "Incomplete List of Mistakes in the Design of CSS" that documents a whole bunch of things which, with the benefit of hindsight, should probably have been named, specified, or implemented differently. The problem is popularity. CSS is so widely used, as soon as something's part of the spec, there will be real websites out there in the real world that rely on it: websites that aren't actively maintained, or very, very hard to update. . . if you've bought a broadband router, or a printer, or a network camera in the last twenty years, it's probably got a web interface built-in. I've got bits of gear on my home network that are so old the web interface was designed to work with Internet Explorer 6. . . fortunately, the maintainers of web standards like CSS have worked incredibly hard to make sure that if something worked 20 years ago, it'll still work today, but the downside of that is that CSS - and HTML, and JavaScript - are riddled with historical quirks and things that seemed like a good idea at the time.

And finally, and probably because of all the reasons we just talked about: most CSS you'll find in the wild is terrible. Development teams focus on features, data,

performance; the CSS gets bolted on as an afterthought, hacked around using increasingly complex rules until it sort of mostly looks good enough... then the next developer comes along, can't make any sense of what's already there, and every time they modify it somebody complains that it's broken something on a completely different part of the website, so instead of being evolved and maintained over time, we just keep adding new layers on top of everything that's already there, until you've got a site with ten different kinds of buttons on it and nobody knows what'll break if you change one.

Now, if you're one of the folks for whom all of the above feels painfully true, I have bad news: CSS isn't going away. Ever. It's one of the fundamental building blocks of the web. You can write your backend and your microservices in anything you like - Java, C#, PHP, Go - but when it comes to end users, the best way to deliver your solution is the web, and the web runs on HTML, JavaScript, and CSS. Even desktop applications these days are often built using frameworks like Electron, which rely on CSS for styling and interaction design.

There is good news too, though - and the good news is that CSS is actually pretty awesome now. If, like a lot of developers, you learned a bit of CSS early on in your career and haven't really gone back to it, you're in for all kinds of fun. We've got grids and flexbox layouts, animations, keyframes, transformations... CSS even has variables now. A lot of layouts and designs that used to be really complicated are now fairly straightforward - and those new features also mean we can build all kinds of new interfaces and capabilities into our web applications. Sound good? Alright, let's kick things off.

Semantic HTML

OK, first things first. As we've already seen, CSS, on its own, doesn't do anything. You can't build an application using only CSS; to understand how CSS works, you need to understand its relationship to HTML - the Hypertext Markup Language.

Start with the basics. Almost every website and application is going to draw stuff on your screen. Words and images. The words might be there for you to read them - headings, articles, paragraphs - or they might be parts of the user interface: labels, menus, drop-down lists. Images might be content - diagrams and photographs - or they might be icons, buttons, or other interactive elements.

HTML is how we control the structure of all that content. It gives us a way to organise content into containers, to sort and group the various bits of content that make up our website or application - and, by default, it applies some very basic visual styling to that content.

There have been numerous "official" versions of HTML over the years, going all the way back to HTML 1.0 in the early 1990s, and for a long while those versions were maintained and managed by the W3C, the World Wide Web Consortium. The most recent numbered version was HTML5, published in 2014 - but along the

way, Apple, Mozilla and Opera announced that they were going to collaborate on a different approach to evolving HTML, via something called the Web Hypertext Application Technology Working Group - or the what-double-you-gee, or the what-wee-gee, or the what-wig, depending on who you're talking to.

Consequently, HTML no longer has version numbers: instead, it's controlled by something called the HTML Living Standard. This is a continuously-evolving document that describes how HTML works: folks like us can use it to look up HTML tags, attributes and syntax, and the folks who build web browsers use it as a reference for making sure their browser engines will render those same tags, attributes and syntax properly.

Now, remember that the web is a moving target. At any given point, there's a bunch of features out there which will work in some browsers, but aren't yet supported in others. There's things which are coming soon, which you can opt in to if you want to try them out; there's things which are deprecated, and so you probably shouldn't use them in your projects - but they, mostly, still work. With a handful of exceptions, most notably Java applets and plugins like Shockwave and Flash, everything that *has* ever worked on the web will *still* work on the web; it's just there might be a better way to do it now.

Let's create a really simple web page, just to remind ourselves of the basics - and to establish a few conventions that we'll be using throughout the rest of the course.

```
{% example index.html %}
```

Our page opens with a DOCTYPE declaration. This tells the browser that we're sending it modern HTML - it was introduced as part of HTML 5, and at the time, if you didn't include this DOCTYPE declaration, browsers would fall back to using a legacy rendering model sometimes known as "quirks mode", which meant they could include support for the latest features and standards - activated by the DOCTYPE - but still render older pages and sites. It's not such a big deal any more out on the open web, where the vast majority of sites use relatively modern code, but there's a lot of corporate intranets and embedded systems out there which still rely on quirks mode.

Next up, HTML. You know what this does... `<html>` marks the start of a web page, and `</html>` marks the end of it. We say that `<html>` is the opening tag, that `</html>` is the closing tag, and that the entire chunk of content, including the opening and closing tags, is an HTML element. Well, actually, everything on the page is an HTML element, but this one's the HTML HTML element. Stick with me. It'll make sense.

Next up, the `<head>` element. Head is, mostly, not there for humans; it's for computers. It's where we put metadata, links, scripts - and, in this example, the page title, using the HTML `<title>` element, which shows up in the browser tab, bookmarks, and a handful of other places - but notice that the page itself is still blank. Let's fix that.

The `<body>` element is the actual page - the bit that's there for humans to read. Starts with `<body>`, ends with `</body>`, and by default, the browser's going to render everything in between - so let's give it something to render.

`<h1>` denotes a level 1 heading - the biggest, most important heading on a page. Hello World. Then we'll throw in a paragraph tag - HTML is totally awesome - and we'll wrap the word totally here in `em` tags, short for *emphasis*.

Finally, let's plug in another paragraph. This one contains an `input` tag, which has a couple of what are called *attributes*. One attribute tells us what type of input it is - `type="button"` - and the other tells what the button's value is - `value="Click me!"` .

Now, notice that the `input` tag doesn't have a closing tag. Input tags are special - they're what's technically known as a *void tag*, or sometimes an *empty* or a *self-closing* tag. You'll sometimes see code in the wild where these kinds of standalone tags have a trailing slash inside the tag itself - like this: `<input />`. Now, I'm gonna put my hand up here: I've been writing tags like that for literally decades, and it's never broken anything... but when I was doing the research for this course, I found out that apparently we don't do that any more. See, there was a project in the early 2000s called XHTML; in the early days of web services, everybody got very excited about XML as a data format and decided it would be an awesome idea to come up with a much stricter version of HTML that could work with software designed to process XML, and so XHTML was born. A bunch of folks like me got very excited about it, mainly 'cos you could run it through an XML validator and it'd tell you whether your page was valid XML or not... but it turned out that wasn't actually very important, and XHTML introduced a bunch of unnecessary complexity - like case sensitive attributes - that mostly just made things harder. You'll still find XHTML out there - it's supported by just about every web browser out there, and is apparently used in a lot of content management systems - but we're going to stick to regular, non-XML-flavoured HTML in this course, and that means no closing slash inside standalone tags.

The main reason I brought that up is that there's a huge amount of stuff out there on the web that developers like me have picked up over the years... and in a lot of cases, it's stuff that used to be really important, so we got into the habit of doing it everywhere, and then browsers evolved, and one day it wasn't necessary any more, but because it doesn't break anything, folks just keep on doing it. In this course, I've tried really hard to reflect the current state of the web, but I can't promise one or two historical idiosyncrasies won't accidentally sneak in.

Anyway. There it is: a web page. Now, If this is the first time you've seen anything like this... hey, welcome to the web; you're going to love it here. But I'm guessing the vast majority of you folks out there have seen this kind of thing before. The reason I wanted to go through it like this is to get you all up to speed on a couple of really important ideas and concepts - and to introduce you

to the terminology we're going to be using in this course, so when I talk about elements and tags and attributes and values, you know what I mean.

You'll sometimes hear folks refer to HTML, CSS and JavaScript as the three pillars of the web: HTML is about structure, CSS is about appearance, and JavaScript is about behaviour. I think that's a great way to look at it, because it helps me figure out how to approach solving certain kinds of problems. If I'm concerned about what something *means*, look at the HTML. If I'm concerned about what something *looks like*, look at the CSS, and if I'm concerned about what it *does*, I'm probably going to end up writing JavaScript.

Except... it's not quite that clear-cut. Take a look at our page here. We haven't written any CSS yet, but we've got three different kinds of text here... the heading 1, that's in Times New Roman for me, and it's kinda chunky. The paragraph is, well, ordinary text; that emphasis bit is in italics, and the button... that's in a completely different font altogether, and it's got a border, and a grey background - at least, it does here on my Windows machine - and, well, it looks like a button.

What's happening here is that the HTML is telling the browser "hey, this thing here is a heading - make it look important", and the browser is applying a default, built-in stylesheet which says "hey, heading 1? That's important, so make it big and bold". The emphasis tag - we didn't *say* "please draw this in italics", we said "please emphasise this", and the browser is interpreting that by drawing it in italics. And the button? We just said "make it a button" - something the user's going to look at and go "hey, that looks clickable!" - and folks who use Windows learned long ago that if something looks like that, grey with raised edges? You can probably click on it.

This is **semantic HTML**. The HTML doesn't say "make this big, make this bold, make this blue" - it says "this is a heading, this bit needs emphasis, this is something the user can interact with"; the document structure is built around semantics - what things *mean* - rather than what they *look like*. A human being will glance at this page and go "oh, big bold heading - clearly that's what this document is all about." But something like a screen reader, or a search engine, or even an AI agent, can also tell just from looking at the markup that, yep, there's a single H1 heading on this page, so that's probably what the document is about. We'll be talking about semantic HTML a lot more as we explore various applications and layouts during the rest of the course.

Introducing Web Accessibility

Something else to bear in mind as we get further into all this: not all browsers are the same. For a long while in the early days of the web, the two main browsers, Netscape and Internet Explorer, had a lot of fundamental incompatibilities, so for anything too complex you'd end up having to maintain two different versions of your JavaScript code. We called this the "browser wars" - and fortunately

the browser wars are ancient history now; all of the mainstream web browsers out there these days have excellent support for modern web standards.

What we have seen, though, is a proliferation of devices. Somebody could be viewing your web page on a laptop, a mobile phone, a smartwatch, a desktop computer with a 52" ultrawide display - and so as we start to think about styling and layout, we've got to consider what'll happen to our pages across a whole range of screen layouts and sizes.

We've also got to remember that the user controls their own browser. If they want bigger fonts, easy - they can go into Settings, Appearance, Font size, and whack that up to Large, or Extra Large. If they're fed up of Times New Roman, they can go into Customise Fonts and set their standard font to Arial, or Calibri, or even Comic Sans. It's enlightening to browse your usual favourite websites with the font size cranked up. Some sites - like the BBC News site - handle this extremely well, because the whole layout is based on the base font size, so everything else expands to compensate. Some sites - like Reddit - override the defaults with their own hard-coded font size, so changing the browser settings doesn't actually do anything. And some sites - like Gmail - will respect the font size, but don't adjust their layout to compensate so things end up looking really weird. And then there's Page Zoom, which just lets you make the whole page bigger and smaller.

What actually ends up on your user's screen is going to be a compromise between your site's design, and the end user's settings and preferences - and that's important. Accessibility is a big deal, and a key part of making your online content accessible is giving your end users control over the things they might need to change. If text is too small, they should be able to make it larger. If you're presenting important information via diagrams or images, provide an alternative for people who can't see those images.

One thing I want to mention while we're here: for as long as I've been developing websites, people have been talking about using tags like **strong** and **em** - instead of just making text bold or italic - because, in theory, something like screen reader software could emphasise those elements. Sounds like a good idea, right? And nobody's going to argue *against* that... but, unfortunately, it isn't actually true. Let's add a couple more paragraphs to our example page here - let's have one that's just a regular paragraph, our existing one that uses the **em** tag, and a third one that uses a **strong** tag. Now I'm going to open up the NVDA screen reader software, and listen to how it reads out that content:

That's not a case *against* semantic markup - in fact, using well-structured markup is an excellent starting point when it comes to making web content accessible. But the example of **em** tags and screen readers is so ubiquitous that I thought it was worth pointing out that it's Not Actually A Thing.

Review & Recap

In this section, we’ve discussed why CSS is so unpopular with developers:

- CSS uses a declarative programming model, not a procedural one: you don’t provide instructions, you describe results.
- CSS is closely associated with visual design, and so books and training material often treat CSS as part of a design toolkit rather than an engineering capability
- CSS is hard to test; bugs and regressions will often go unnoticed until somebody complains that “the website is broken”
- There’s a lot of very bad CSS code in the wild; many of us have ended up having to work with it at some point in our career, and that contributes a lot of negativity to developers’ perception of CSS as a language.

Properties and Values

CSS, fundamentally, allows us to do three things:

1. Target an element, or elements, on a web page.
2. Choose a property which is relevant to that element.
3. Assign that property to a value, which controls how the browser will display it.

The simplest way to do this is what’s called **inline CSS**. We’re going to find an element on our page - say, that heading 1 - and we’re going to add a **style** attribute containing a property - **color** - and a value - **green**. You can probably figure out what that means, right? Yep, draw that heading using the colour green.

I’m British, so in prose, I write *colour*, with a *u*. CSS was invented in Switzerland, by Håkon Wium Lie, who is Norwegian, and Bert Bos, who is Dutch, and was heavily influenced by HTML - created by Tim Berners-Lee, who is also British. So naturally CSS uses the American spelling **color**, without a *u*.

In this course, I call it *colour* when it’s in prose, and **color** when it’s in code or referring to the CSS keyword. Just be grateful nobody’s inquiring about using fibre to serialise their aluminium licence.

The keyword **color** is one of literally hundreds of properties defined by the CSS language. There are properties for colors, fonts, size, layout, position, animation - and we’re going to meet just about all of them in this course. The value **green** is one of the 148 named colours defined in the current CSS specification – and as we’ll see later, named colours is just one of several different colour systems supported by CSS.

And that rule gets applied to the H1 element because we’ve declared it inline, using the **style** attribute. So far, so good. But what if we wanted all our paragraphs to be purple?

Sure, we could add a **style** attribute to every paragraph... copy, paste, paste... but that's not ideal. There's a principle in software engineering called DRY - Don't Repeat Yourself; the idea that every decision about your application should be reflected in your code in exactly one place. We've decided to make our paragraphs purple - that's one decision - and at the moment, it's reflected in our code in three different places, which is bad.

Introducing Selectors

Let's meet CSS selectors. Instead of putting our styles inline, we're going to add a **style** element to the **head** of our page. This lets us define rules that apply to everything on the page, but to do that, we need to add something called a **selector**, which tells the browser which element, or elements, each rule applies to.

Let's add a selector **p**, which matches any paragraph element, then open braces, put in our rule - **color: purple** - and close braces. That's a CSS rule: paragraphs on this page should be purple.

You'll notice that the **em** and the **strong** elements there have also changed colour, but the text on the button hasn't. This is because of the **C** in CSS, which stands for **cascading**. What's actually happening here is that we've defined a rule saying "stuff in paragraphs should be purple", but the browser has a default rule that says "buttons should have black text on a grey background", and in this situation, the browser rule about buttons takes precedence over our new rule about paragraphs. We'll talk a lot more about this when we get to the part about **specificity**, because understanding how the browser decides which rules to apply, and which rules to ignore, is a vitally important part of understanding how CSS actually works.

OK, so: inline styles? Great if you only need to use something once. Putting a style block in the header? Great if you only need to use styles on one page. But it turns out most web applications have more than one page - even the ones that call themselves single-page apps - and so what most sites do is to put all the CSS in a separate file and then import that using an HTML **link** tag

Create a new file, **styles.css**, and move the contents of that **style** block into the new file.

```
{% example styles.css %}
```

Then remove the empty **style** element, and replace it with a **<link>** element:

```
{% example index.html mark_lines="6" %}
```

The **<link>** element has two attributes:

- **rel** is short for *relationship* - in this case, "**stylesheet**" indicates that the linked file contains CSS rules to apply to the page
- **href** is the URL of the file to load.

Another historical quirk: you'll often see `link` elements include an extra attribute: `type="text/css"` - a relic of the days when servers would serve CSS files with the wrong MIME type and browsers weren't smart enough to ignore it.

We've now got a page which will load CSS rules from a separate file - and that's a big deal, because we can have many pages which load their CSS rules from that same file, and so our CSS is now reusable. We're not writing stylesheet rules for each page, we're building a stylesheet that applies to our entire website.

CSS Comments

Like most programming languages, CSS lets us put comments in our code - either to explain what's going on, or to temporarily disable a particular rule or selector.

CSS only has a single style of comment - a comment starts with `/*` and ends with `*/`, and comments can't be nested.

```
/* An example of a CSS comment */
p {
  color: green;
  /* font-size: 200%; */
}
```

Remember, the point of comments in CSS - and in any language - isn't to tell us what the code does. We can figure that out by reading it. Here's an example of a bad comment:

```
/* Make all paragraphs large */
p {
  font-size: large;
}
```

...we can tell that's going to make paragraphs large, because we can read the CSS rule! A good comment tells the next person to read the code something useful which they can't infer just from reading it:

```
h1 {
  /* Using actual black here causes weirdness with dark mode, so we use
   a very, very dark grey color instead. */
  color: #010101;
}
```

Review & Recap

In this section, we learned about the core structure of CSS, how the CSS language actually works, and how browsers read CSS rules when they're rendering a web page.

- CSS is built around three basic units of syntax:
 - **selectors** control which elements are affected by a rule
 - **properties** control which aspect of those elements we're modifying
 - **values** are what actually changes it.
- CSS rules can be:
 - **Inline**, inside a **style** attribute that's part of the HTML tag
 - **Internal**, contained in a **<style>** element in the **<head>** of a document
 - **External**, loaded from a separate file using a **<link>** element.